

01-05 后续全部 PPT 复习讲义

目录

目录	7
1. 软件工程基础与项目管理	7
1.1 软件是什么	7
1.2 软件的典型特征	8
1.3 软件工程的定义	8
1.4 软件危机与 Ariane 5 案例	9
1.5 SWEBOK 知识域	9
1.6 项目与项目管理	10
1.7 项目管理五大过程组	10
1.8 风险管理	11
1.9 软件质量保证 SQA	12
1.10 软件配置管理 SCM	12
1.11 变更控制流程	13
2. 需求工程	14
2.1 需求工程的含义	14
2.2 问题域	14
2.3 需求层次	15
业务需求	15
用户需求	15

系统级需求	16
2.4 需求类型	16
功能需求	16
非功能需求	16
外部接口需求	17
约束	17
数据需求	18
2.5 AI 辅助需求获取	18
2.6 用例建模	19
基本元素	19
include 与 extend	19
用例粒度	20
Cockburn 层次	20
2.7 用例描述模板	20
2.8 用户故事与 BDD	21
2.9 需求分析模型	22
2.10 概念类图	22
从文本提取概念类	23
关联与多重性	24
2.11 顺序图	24
2.12 状态图	25
2.13 OCL	26
2.14 SRS 与需求验证	27
2.15 好需求的标准	27
2.16 需求追踪	28
2.17 歧义与冲突	29
2.18 Spec 驱动开发	30
3. 软件体系结构与人机交互	30

目录	3
3.1 设计的任务	30
3.2 设计层次	31
高层设计：体系结构设计	31
中层设计	31
低层设计	32
3.3 体系结构的组成	32
3.4 4+1 视图	33
3.5 体系结构风格	33
分层风格	33
MVC	34
3.6 体系结构设计过程	34
3.7 接口与依赖倒置	35
3.8 体系结构验证	36
3.9 人机交互设计	37
3.10 人的认知特点	37
3.11 Fitts 定律	37
3.12 用户类型	38
3.13 直接操纵	38
3.14 导航与反馈	39
3.15 黄金规则与 Nielsen 启发式	39
3.16 HCI 设计过程与评估	40
4. 详细设计、模块化与信息隐藏	41
4.1 详细设计的范围	41
4.2 责任与协作	42
4.3 详细设计过程	42
4.4 GRASP 原则	43
Low Coupling 低耦合	43
High Cohesion 高内聚	43

目录	4
Information Expert 信息专家	43
Creator 创建者	44
Controller 控制器	44
4.5 控制风格	44
集中式控制	44
委托式控制	45
分散式控制	45
4.6 Parnas 信息隐藏	45
4.7 模块化	45
4.8 耦合	46
访问耦合	46
Law of Demeter	47
4.9 接口隔离与面向接口编程	47
4.10 继承耦合	48
4.11 LSP	49
4.12 组合优于继承	49
4.13 封装	50
4.14 设计原则 P1-P14	50
4.15 集成测试与 Mock	51
5. 设计模式	52
5.1 设计模式的构成	52
5.2 Strategy 策略模式	52
5.3 Abstract Factory 抽象工厂	54
5.4 Factory Method 工厂方法	55
5.5 Singleton 单例	55
5.6 Iterator 迭代器	56
6. 代码设计与软件构造	57

6.1 代码设计关注点	57
6.2 布局	57
6.3 命名	58
6.4 注释与 Javadoc	59
6.5 AI 代码生成 Prompt	59
6.6 软件构造	60
6.7 构造技术	60
6.8 代码审查	61
6.9 集成与构建	62
6.10 构造管理	62
6.11 坏味道与重构	63
6.12 TDD	64
6.13 AI 编程原则	64
7. 综合题答题模板	65
7.1 分层接口代码模板	65
7.2 校园卡自助系统物理包图文字版	68
7.3 充值服务接口题模板	70
7.4 需求题答题模板	72
写业务需求	72
写用户需求	72
写系统级功能需求	72
写非功能需求	73
7.5 概念类图题模板	73
7.6 顺序图题模板	74
7.7 状态图题模板	75
7.8 设计原则题模板	75
7.9 设计模式题模板	76
7.10 代码质量题模板	77

7.11 HCI 题模板	77
7.12 配置管理题模板	77
8. 快速背诵清单	78
软件工程	78
项目管理	78
需求工程	78
体系结构	79
HCI	79
详细设计	79
设计模式	79
构造	80

范围：01-01 到 04-02，以及 设计模式、代码设计、软件构造。用法：先背“概念与原则”，再练“场景题模板”。考试题通常不会只考名词解释，而是要求把原则落到需求、架构、接口、详细设计、代码质量和测试上。

目录

1. 软件工程基础与项目管理
 2. 需求工程
 3. 软件体系结构与人机交互
 4. 详细设计、模块化与信息隐藏
 5. 设计模式
 6. 代码设计与软件构造
 7. 综合题答题模板
-

1. 软件工程基础与项目管理

1.1 软件是什么

软件不是单纯的程序。课程中的核心表达是：

软件 = 程序 + 数据 + 文档 + 知识

IEEE 610.12-1990 对软件的定义强调三类内容：

- 程序
- 规程或过程
- 相关文档与数据

答题时要写出“软件不只等于代码”。代码只是可执行部分，文档、数据、过程和领域知识共同决定软件能否被理解、运行、维护和演化。

1.2 软件的典型特征

常见概括：

- 无形：软件没有物理形态，进度和质量不易直接观察。
- 复杂：状态、分支、交互和依赖数量高。
- 可复制：复制成本低，但开发和维护成本高。
- 易修改：修改入口多，但影响传播复杂。
- 退化：软件不会物理磨损，但会因环境变化、需求变化和补丁叠加而退化。
- 依赖：依赖硬件、操作系统、网络、数据库、第三方库和组织流程。
- 智力密集：核心产出来自设计、抽象、分析和协作。

考试答“软件危机”时，要把这些特征与项目失败联系起来：规模变大后，靠个人经验写代码无法稳定控制质量、进度、成本和维护。

1.3 软件工程的定义

IEEE 定义的关键词：

- 系统化
- 规范化
- 可度量
- 开发、运行和维护

可写为：

软件工程是把系统化、规范化、可度量的方法应用于软件的开发、运行和维护，并研究这些方法的学科。

软件工程解决的不是“怎么写出一段代码”，而是“如何在多人协作、长期演化、质量受控的条件下交付软件系统”。

1.4 软件危机与 Ariane 5 案例

软件危机的标志性节点是 1968 年 NATO 软件工程会议。

常见问题：

- 项目延期
- 成本超支
- 软件质量低
- 难以维护
- 文档缺失
- 需求不断变化
- 开发过程不可控

Ariane 5 案例的答题重点：

- 代码本身来自已有系统，不等于在新环境中正确。
- 隐含的数值范围假设没有被充分记录和验证。
- 复用没有配套完成需求、环境、接口和异常条件验证。
- 失败属于需求假设、集成验证、配置和变更管理问题，不只是编码问题。

1.5 SWEBOK 知识域

传统知识域覆盖需求、设计、构造、测试、维护、配置管理、工程管理、过程、模型方法、质量、职业实践、经济学、计算基础、数学基础、工程基础等。

SWEBOK V4 增强关注：

- 软件体系结构
- 软件工程运维
- 软件安全
- AI for SE：用 AI 辅助软件工程
- SE for AI：用软件工程方法建设 AI 系统

答题时可以用一句话收束：

AI 增强了需求整理、设计草图、代码生成和审查效率，但工程纪律仍由需求验证、接口约束、测试、配置管理和人工评审保证。

1.6 项目与项目管理

PMBOK 中项目的核心特征：

- 临时性：有明确开始和结束。
- 独特性：交付独特产品、服务或成果。

软件项目管理要控制三个基本约束：

- 范围
- 时间
- 成本

三者共同影响质量。范围扩大、时间压缩或成本不足，都会传导到质量风险。

1.7 项目管理五大过程组

按课程表达：

1. 项目启动
2. 项目计划
3. 项目执行
4. 跟踪与控制
5. 项目收尾

常见管理活动：

- 项目计划

- 团队管理
- 成本管理
- 软件质量保证
- 度量
- 过程管理
- 进度管理
- 风险管理
- 配置管理

1.8 风险管理

风险管理循环：

1. 风险识别
2. 风险分析
3. 优先级排序
4. 风险应对
5. 风险监控

常见风险类型：

- 技术风险：新框架、新平台、性能瓶颈、兼容性问题。
- 人员风险：关键成员离开、经验不足、沟通失败。
- 需求风险：需求不清、范围膨胀、利益相关者冲突。
- 进度风险：估算失真、依赖延期、任务拆分不合理。
- 外部风险：政策、供应商、硬件、网络、第三方接口变化。

答题模板：

先列风险，再写影响，再写应对措施。例如：第三方支付接口变更属于外部和技术风险，会影响充值功能上线；应对措施是提前建立接口契约、Mock 服务、回归测试和变更通知机制。

1.9 软件质量保证 SQA

SQA 关注“过程和产品是否满足质量要求”。

常见活动：

- 代码审查
- 静态分析
- 测试
- 过程审计
- 质量度量
- 阶段评审

里程碑质量活动：

- 需求阶段：需求评审、需求度量。
- 体系结构阶段：架构评审、集成测试策略。
- 详细设计阶段：设计评审、设计度量、集成测试准备。
- 构造阶段：代码审查、代码度量、单元测试。
- 测试阶段：缺陷度量、覆盖率分析、回归测试。

1.10 软件配置管理 SCM

SCM 的目标：

- 识别配置项
- 控制变更
- 记录和报告配置状态
- 审计配置
- 保证产品与规格一致

核心术语：

- 配置项 CI：纳入配置管理的项目制品，如需求文档、设计文档、源代码、测试脚本、部署脚本、数据库脚本。
- 基线 Baseline：经过正式评审和批准的规格或产品，后续变更必须走正式变更控制。

SCM 常见活动：

1. 配置项识别
2. 版本管理
3. 变更控制
4. 配置审计
5. 状态报告
6. 发布管理

1.11 变更控制流程

标准作答流程：

1. 提交变更请求 CR。
2. 进行影响分析：影响需求、设计、代码、测试、进度、成本和风险。
3. 进行风险评估。
4. CCB 审查并批准或拒绝。
5. 制定回滚计划。
6. 执行变更。
7. 更新基线。
8. 上线验证。
9. 更新状态报告。

Knight Capital 案例的答题重点：

- 部署遗漏一台服务器。

- 新旧代码路径并存导致异常交易。
 - 这是发布管理、配置管理和上线验证失败。
 - 技术错误背后是过程控制缺失。
-

2. 需求工程

2.1 需求工程的含义

需求工程覆盖所有围绕需求展开的活动。

核心活动：

- 需求获取
- 需求分析
- 需求规格说明
- 需求验证
- 需求管理

需求工程不是简单“问用户要功能”。它需要理解问题域、利益相关者、业务流程、约束条件、数据规则和外部系统。

2.2 问题域

问题域包含：

- 业务目标
- 业务流程
- 领域术语
- 组织规则
- 相关人员
- 现有系统

- 数据对象
- 外部约束

需求工程的关键是先理解问题域，再定义系统边界和解决方案。

2.3 需求层次

业务需求

回答“为什么建设系统”。

关注：

- 战略目标
- 业务价值
- 愿景
- 范围

示例：

建设校园卡自助系统，提高充值、挂失、查询等业务的自助化比例，降低人工窗口压力。

用户需求

回答“用户想用系统完成什么任务”。

特点：

- 面向用户任务
- 语言不够精确
- 常混合功能、界面和约束

示例：

学生希望在网页或移动端自助完成校园卡充值，并查看充值记录。

系统级需求

回答“系统必须提供什么能力和约束”。

特点：

- 精确
- 可验证
- 可追踪
- 面向系统交互和响应

示例：

系统接收充值请求后，应校验校园卡状态和支付结果；支付成功后创建充值记录并更新账户余额。

2.4 需求类型

功能需求

说明系统做什么。

示例：

- 用户登录。
- 查询校园卡余额。
- 提交充值订单。
- 查询充值记录。
- 挂失校园卡。

非功能需求

说明系统达到什么质量或约束。

常见维度：

- 性能

- 安全
- 可用性
- 可靠性
- 可维护性
- 可移植性
- 易用性

性能维度：

- 速度
- 容量
- 吞吐量
- 负载
- 时效性或实时性

外部接口需求

说明系统与外部系统、硬件、用户界面之间的交互约束。

示例：

- 与支付平台通过 HTTPS 通信。
- 与校园卡中心系统同步余额。
- 前端通过 REST API 调用后端服务。

约束

说明系统建设必须遵守的限制。

示例：

- 使用学校统一身份认证。
- 数据库存储在校内数据中心。
- 后端采用 Java Spring Boot。

数据需求

说明系统需要管理的数据对象、数据字段、完整性规则和保留策略。

示例：

- 充值记录包含记录编号、校园卡号、金额、支付渠道、支付状态、创建时间。
- 充值金额必须大于 0。

2.5 AI 辅助需求获取

AI 适合：

- 从访谈记录中提取功能点。
- 整理用户故事。
- 生成需求文档初稿。
- 把自然语言改写成结构化条目。
- 检查编号、格式和冲突线索。

AI 的薄弱点：

- 容易遗漏隐含业务规则。
- 容易忽略系统边界。
- 容易生成不符合真实组织流程的功能。
- 对领域例外、权限规则、历史系统接口理解不足。

推荐流程：

1. Prompt：给出背景、角色、范围、输出格式。
 2. Generate：生成需求条目或模型草图。
 3. Review：人工核对业务规则、边界、异常和术语。
 4. Refine：补充约束、优先级、验收标准和追踪关系。
-

2.6 用例建模

用例描述系统与外部对象之间的一组交互行为。它从用户目标出发，描述系统如何向参与者提供有价值的服务。

Cockburn 的区分：

- 用例：包含一个用户目标下的所有场景。
- 场景：用例中的一条具体路径。

基本元素

- 参与者 Actor：系统外部与系统交互的角色。
- 用例 Use Case：系统提供的服务。
- 系统边界：区分系统内部和外部。
- 关系：关联、包含、扩展、泛化。

参与者可以是人、外部系统、硬件设备或定时器。参与者位于系统边界外，用例位于系统边界内。

include 与 extend

include：

- 基本用例每次执行时都要执行被包含用例。
- 方向：基本用例指向被包含用例。
- 用于抽取多个用例共享的公共行为。

extend：

- 扩展用例在特定条件下插入基本用例。
- 方向：扩展用例指向基本用例。
- 用于异常、可选、附加行为。

示例：

- “充值” include “身份验证”：每次充值都要验证用户身份。
- “充值失败处理” extend “充值”：只有支付失败或写入余额失败时触发。

用例粒度

合格用例通常满足：

- 对应一个业务事件。
- 由参与者发起。
- 在连续时间内完成。
- 对参与者有业务价值。

常见错误：

- 把增删改查拆成太细的技术操作。
- 把同一业务目标拆散。
- 把登录、数据校验、数据库连接当作独立业务用例。
- 在用例图里写实现细节。

Cockburn 层次

- 概要层：范围太大，适合说明系统目标。
- 用户目标层：最适合作为主用例。
- 子功能层：粒度较小，常用于 include 或内部步骤。

2.7 用例描述模板

十项模板：

1. 用例编号
2. 用例名称
3. 参与者
4. 触发条件
5. 前置条件
6. 后置条件

- 7. 基本流程
- 8. 扩展流程
- 9. 特殊需求
- 10. 备注

前置条件是用例开始前已经成立的系统状态，不是“用户输入账号密码”这类步骤。

示例：充值用例简表

项目	内容
用例编号	UC-Recharge
用例名称	校园卡充值
参与者	持卡人
触发条件	持卡人选择充值功能
前置条件	持卡人已通过身份认证，校园卡状态正常
后置条件	充值成功时账户余额增加，并生成充值记录
基本流程	选择充值金额；系统创建充值订单；调用支付接口；支付成功后更新余额；显示结果
扩展流程	支付失败时记录失败原因并提示用户；校园卡状态异常时拒绝充值
特殊需求	充值结果写入和记录生成保持一致

2.8 用户故事与 BDD

用户故事常用格式：

作为 < 角色 >，我想要 < 能力 >，以便 < 业务价值 >。

示例：

作为持卡人，我想要在线充值校园卡，以便在食堂和门禁场景继续使用校园卡。

BDD 使用 Given-When-Then：

Given 持卡人已登录且校园卡状态正常
When 持卡人提交 100 元充值并完成支付
Then 系统增加校园卡余额并生成一条成功充值记录

ATDD 强调先定义验收测试，再驱动设计和实现。

2.9 需求分析模型

常用模型：

- 用例图
- 概念类图
- 顺序图
- 状态图
- ER 图
- OCL 约束

面向对象需求分析输出：

- 用例图和用例描述
- 概念类图或领域模型
- 顺序图
- 状态图
- OCL 约束

2.10 概念类图

概念类图描述问题域中的概念及其关系，属于分析模型。

它与设计类图的区别：

项目	概念类图	设计类图
阶段	需求分析	详细设计
关注	领域概念	软件类、接口、方法
内容	类名、属性、关联、多重性	属性、方法、可见性、接口、继承、依赖

项目	概念类图	设计类图
来源	用例、业务文本、领域知识	架构、设计原则、实现技术

从文本提取概念类

步骤：

1. 收集用例描述、访谈记录和业务规则。
2. 标记名词和名词短语。
3. 合并同义词。
4. 删除纯属性、实现概念、外部系统、重复词和临时消息。
5. 与业务人员确认剩余概念。
6. 为概念添加属性、关联和多重性。

判断一个词能否成为概念类：

- 系统需要长期记录它。
- 它拥有多个属性。
- 它参与业务规则。
- 它与其他概念存在稳定关系。

示例：

- “充值记录”是概念类，因为系统要保存记录编号、金额、支付状态、时间等属性。
- “金额”通常是属性，不是概念类。
- “支付平台”是外部系统，在系统边界外。

关联与多重性

动词短语常提示关联：

- 用户拥有校园卡。
- 校园卡产生充值记录。
- 充值记录对应支付订单。

数量词提示多重性：

- 一个用户可以拥有多张校园卡。
- 一张校园卡可以有多条充值记录。
- 一条充值记录对应一个支付订单。

2.11 顺序图

顺序图描述对象之间按时间顺序发生的消息交互。

画法要点：

- 先选一个用例。
- 从基本流程开始。
- 每个交互步骤对应消息。
- 使用 `alt` 表示分支，使用 `opt` 表示可选步骤。
- 消息顺序必须与业务流程一致。

充值用例的典型对象：

- 持卡人
- 充值界面
- 充值控制器
- 充值服务
- 校园卡仓储

- 充值记录仓储
- 支付网关

典型流程：

1. 持卡人提交充值请求。
2. 控制器调用充值服务。
3. 服务查询校园卡状态。
4. 服务创建充值记录或充值订单。
5. 服务调用支付网关确认支付结果。
6. 服务更新余额。
7. 服务保存充值记录。
8. 控制器返回充值结果。

2.12 状态图

状态图描述一个对象在生命周期内的状态变化。

适合建模：

- 订单
- 校园卡
- 支付记录
- 设备
- 会话

充值记录状态示例：

- Created
- Paying
- Success

- Failed
- Cancelled

状态图可以直接指导测试：

- 每条合法迁移对应一个测试。
- 每条非法迁移对应一个异常测试。

2.13 OCL

OCL 是附着在 UML 模型上的声明式约束语言。

特点：

- 无副作用。
- 说明不变式、前置条件和后置条件。
- 可表达集合约束。

常见元素：

- `inv`：不变式。
- `pre`：前置条件。
- `post`：后置条件。
- `@pre`：操作执行前的值。
- `forAll`、`exists`、`select`、`collect`：集合操作。

示例：

```
context Recharge
inv PositiveAmount:
    self.amount > 0

context CampusCard::recharge(amount)
pre ValidAmount:
    amount > 0
post BalanceIncreased:
    self.balance = self.balance@pre + amount
```

2.14 SRS 与需求验证

SRS 是系统级需求规格说明，从产品和系统视角列出需求。

用例文档和 SRS 的区别：

项目	用例文档	SRS
视角	用户交互	系统能力
组织方式	按用户目标	按功能、接口、数据、质量属性
重点	场景和流程	可验证、可追踪的需求条目

SRS 条目要使用刺激-响应结构：

当 < 刺激 > 发生时，系统应 < 响应 >。

示例：

当持卡人完成支付且支付平台返回成功结果时，系统应更新校园卡余额并生成成功充值记录。

2.15 好需求的标准

常见标准：

- 正确性
- 完整性
- 一致性
- 可读性
- 可修改性
- 可验证性
- 可追踪性

验证方法：

- 同行评审

- 原型验证
- 仿真
- 测试用例推导
- 追踪矩阵检查

2.16 需求追踪

前向追踪：

需求 → 设计 → 代码 → 测试

后向追踪：

代码或设计 → 需求来源

追踪矩阵常见列：

- 需求编号
- 需求来源
- 设计模块
- 代码模块
- 测试用例
- 状态

需求属性：

- 编号
- 描述
- 理由
- 来源
- 优先级

- 稳定性
- 状态
- 负责人
- 版本
- 变更历史

2.17 歧义与冲突

歧义类型：

- 词法歧义：同一词有多种含义。
- 句法歧义：句子结构导致多种解释。
- 语用歧义：上下文或业务背景不足导致理解不一致。

冲突类型：

- 直接矛盾：两个需求对同一条件给出相反响应。
- 资源冲突：时间、预算、算力、设备或人员无法同时满足。
- 优先级冲突：不同利益相关者对需求重要性排序不同。

处理流程：

1. 标记冲突需求编号。
2. 说明冲突类型。
3. 找到对应利益相关者。
4. 召开评审。
5. 记录决策和理由。
6. 更新 SRS、追踪矩阵和变更历史。

2.18 Spec 驱动开发

SDD 链路：

Spec → Design → Tasks → Code

好的 Spec 包含：

- 问题背景
- 用户故事
- 功能范围
- 验收标准
- 非功能约束
- 不在范围内的内容
- 待确认问题

在 AI 协作中，Spec 是人与 AI 之间的工程协议。Spec 越清晰，AI 输出越容易被审查、测试和接入项目。

3. 软件体系结构与人机交互

3.1 设计的任务

设计回答“如何实现需求”。

与需求和编程的区别：

阶段	回答问题	关注模型
需求	做什么	用例图、概念类图
设计	如何组织解决方案	包图、组件图、接口、协作
编程	如何用代码实现	类、方法、语句、数据结构

设计管理复杂度的基本手段：

- 关注点分离
- 分解
- 抽象
- 模块化
- 信息隐藏

3.2 设计层次

高层设计：体系结构设计

关注整个系统结构：

- 子系统
- 分层
- 组件
- 部署节点
- 连接方式
- 技术栈约束

常用图：

- 组件图
- 部署图
- 高层包图

中层设计

关注包、模块、类之间如何协作。

常用图：

- 包图
- 交互图
- 设计类图

低层设计

关注单个类、方法、数据结构和算法。

内容：

- 属性
- 方法
- 参数
- 返回值
- 控制结构
- 异常处理

3.3 体系结构的组成

体系结构由三类内容构成：

- 部件或组件
- 连接件
- 配置

解释：

- 组件承担计算、存储或业务职责。
- 连接件定义组件交互方式，如方法调用、REST API、消息队列、数据库连接。
- 配置说明组件和连接件如何组合成整体系统。

体系结构决策一旦落地，修改成本高。错误的层次、错误的依赖方向和错误的部署结构会形成长期技术债。

3.4 4+1 视图

4+1 视图用于从多个角度描述体系结构：

- 逻辑视图：类、包、领域对象、业务职责。
- 开发视图：代码模块、工程结构、组件划分。
- 进程视图：运行时进程、线程、并发、通信。
- 物理视图：服务器、客户端、数据库、网络部署。
- 场景视图：用关键用例串联并验证前四个视图。

答题时常用写法：

使用场景视图验证架构是否支持核心用例；使用逻辑视图说明业务分层；使用开发视图说明代码组织；使用进程视图说明运行时并发和通信；使用物理视图说明客户端、服务器和数据库部署。

3.5 体系结构风格

体系结构风格定义：

- 组件词汇表
- 连接件类型
- 拓扑约束

分层风格

典型三层：

- 表示层
- 业务逻辑层
- 数据访问层

优点：

- 职责清晰。

- 上层不用关心下层实现细节。
- 便于替换界面、业务逻辑或数据存储。
- 便于测试和维护。

缺点：

- 层间调用带来额外开销。
- 简单功能也要穿过多层。
- 层次划分错误会导致绕层访问和依赖混乱。

MVC

MVC 分离：

- Model：数据和业务状态。
- View：界面显示。
- Controller：接收输入并协调模型和视图。

Web 项目中常见对应关系：

- Controller：处理 HTTP 请求。
- Service：执行业务逻辑。
- Repository 或 DAO：访问数据。
- Entity 或 PO：持久化对象。
- DTO 或 VO：接口传输对象或界面展示对象。

3.6 体系结构设计过程

课程中的过程可概括为：

1. 分析需求。
2. 选择体系结构风格。

3. 设计逻辑结构。
4. 设计接口。
5. 设计物理部署。
6. 验证体系结构。
7. 评审并迭代。

答题时不要只画图，要说明为什么选择这个结构。

示例：

企业信息系统强调可维护性、安全性和分工协作，适合采用分层架构。表示层处理用户交互，业务逻辑层封装业务规则，数据访问层封装持久化细节。上层依赖接口而非实现，以降低层间耦合。

3.7 接口与依赖倒置

分层系统中要避免上层直接依赖下层实现类。

推荐结构：

- `service` 包声明业务接口。
- `service.impl` 包提供业务实现。
- `repository` 包声明数据访问接口。
- `repository.impl` 包提供数据访问实现。
- Controller 依赖 Service 接口。
- Service 实现依赖 Repository 接口。

依赖倒置原则：

- 高层模块不依赖低层模块，二者都依赖抽象。
- 抽象不依赖细节，细节依赖抽象。

考试代码题重点：

- 写出接口所在包名。

- Controller 中通过构造器注入 Service。
- ServiceImpl 中通过构造器注入 Repository。
- Controller 不直接访问 Repository。

3.8 体系结构验证

验证关注：

- 核心用例是否能穿过架构完成。
- 层间依赖方向是否正确。
- 接口是否稳定。
- 质量属性是否得到支撑。
- 部署结构是否满足性能、安全和可靠性要求。
- 是否存在架构坏味道。

常见架构坏味道：

- 上层绕过业务层直接访问数据库。
 - 所有业务逻辑堆在 Controller。
 - Service 只有转发，没有业务职责。
 - Repository 混入业务规则。
 - 包之间循环依赖。
 - 共享全局状态过多。
-

3.9 人机交互设计

人机交互关注用户、任务、界面和系统反馈之间的关系。

可用性属性：

- 易学性
- 效率
- 易记性
- 错误率与恢复能力
- 满意度

易学性和效率存在取舍。新手需要清晰入口和提示，熟练用户需要快捷键、批量操作和命令式能力。

3.10 人的认知特点

设计原则：

- 识别优于回忆。
- 图形信息在很多场景下比纯文本更易识别。
- GUI 通过菜单、按钮和图标降低记忆负担。
- CLI 对专家高效，但学习成本高。

3.11 Fitts 定律

Fitts 定律说明：

- 目标越近，操作越快。
- 目标越大，操作越快。
- 屏幕边缘和角落更容易命中。

设计推论：

- 高频按钮要放在易触达位置。
- 危险按钮与常用按钮要保持距离。
- 小图标按钮需要足够点击区域。

3.12 用户类型

- 新手：需要清晰导航、提示和容错。
- 中级用户：需要稳定流程和效率提升。
- 专家：需要快捷键、批处理和可配置操作。

好的界面同时服务不同熟练度用户：入口清楚，操作路径短，反馈及时，并提供快捷方式。

3.13 直接操纵

直接操纵的特点：

- 对象可见。
- 操作可逆。
- 反馈即时。
- 用户有控制感。

优势：

- 学习时间短。
- 错误易发现。
- 用户满意度高。

需要处理的问题：

- 界面隐喻要符合用户心智模型。
- 操作对象和系统状态要清晰。
- 复杂系统需要良好导航结构。

3.14 导航与反馈

导航设计：

- 全局结构：用户知道自己在哪里、能去哪。
- 局部结构：当前任务内的步骤清楚。
- 按任务模型组织，而不是按技术模块组织。

反馈时间：

- 键盘、鼠标、光标反馈要非常快。
- 简单频繁任务在约 1 秒内给出响应。
- 更长操作要显示进度、状态或队列。

3.15 黄金规则与 Nielsen 启发式

三条黄金规则：

- 用户控制。
- 减少记忆负担。
- 保持一致。

Nielsen 十项启发式：

1. 系统状态可见。
2. 系统与现实世界匹配。
3. 用户控制和自由。
4. 一致性和标准。
5. 错误预防。
6. 识别优于回忆。
7. 灵活性和效率。
8. 美学和极简设计。
9. 帮助用户识别、诊断和恢复错误。
10. 帮助和文档。

3.16 HCI 设计过程与评估

常用过程：

1. 理解用户和任务。
2. 建立界面概念模型。
3. 绘制线框图。
4. 制作低保真原型。
5. 用户测试。
6. 制作高保真原型。
7. 可用性评估。
8. 实现并迭代。

评估方法：

- 启发式评估
- 用户测试
- Think-aloud
- A/B 测试
- 眼动分析

AI 交互设计补充：

- 显示 AI 的计划和依据。
 - 允许用户确认、撤销和覆盖。
 - 保留审计日志。
 - 在失败时给出可恢复路径。
 - 通过逐步授权建立信任。
-

4. 详细设计、模块化与信息隐藏

4.1 详细设计的范围

详细设计包含：

- 中层设计：过程、调用关系、类协作。
- 低层设计：数据结构、算法、类型、语句和控制结构。

输入：

- 用例
- 概念模型
- 顺序图
- 状态图
- 非功能需求
- 体系结构模块规格
- 导入和导出接口
- 组件接口

输出：

- 静态结构：类、接口、关系。
- 动态行为：对象协作和消息。
- 状态行为：对象生命周期。
- 实现注解：约束、异常、事务。
- 设计理由：关键选择的依据。

4.2 责任与协作

面向对象详细设计的核心是：

- 责任
- 协作

责任：

- 操作责任：对象要执行的任务。
- 数据责任：对象要维护的信息。

协作：

- 对象之间通过消息调用完成更大的任务。
- 一个对象不直接承担全部工作，而是把子任务委托给合适对象。

委托的意义：

- 降低耦合。
- 分离关注点。
- 让职责落在拥有信息的对象上。

4.3 详细设计过程

可按以下顺序答题：

1. 根据用例和系统事件识别控制入口。
2. 根据概念类图分配数据责任。
3. 根据顺序图分配操作责任。
4. 明确类、接口、属性、方法和关系。
5. 用 GRASP 原则检查职责分配。
6. 用模块化、信息隐藏和设计模式改进结构。
7. 设计单元测试和集成测试。

4.4 GRASP 原则

Low Coupling 低耦合

类之间依赖越少，修改传播越小。

做法：

- 依赖接口。
- 避免全局变量。
- 避免对象链式访问。
- 避免跨层调用。

High Cohesion 高内聚

一个类的职责要集中，成员之间围绕同一目标。

坏例子：

- 一个 Manager 同时处理用户、订单、支付、日志、报表。

好例子：

- RechargeService 处理充值业务。
- PaymentGateway 处理支付平台交互。
- RechargeRecordRepository 处理充值记录持久化。

Information Expert 信息专家

把责任分配给拥有完成该责任所需信息的类。

示例：

- 判断校园卡余额是否足够消费，责任应放在 CampusCard 或管理校园卡余额的服务中，而不是界面对象中。

Creator 创建者

B 创建 A 的条件：

- B 聚合或包含 A。
- B 记录 A。
- B 紧密使用 A。
- B 拥有初始化 A 的数据。

示例：

- `RechargeService` 拥有充值金额、校园卡、支付结果等信息，负责创建 `RechargeRecord`。

Controller 控制器

控制器负责接收系统事件并协调业务对象。

控制器不是所有业务逻辑的堆放点。Controller 接收请求并调用 Service；Service 承担业务流程；领域对象承担自身规则。

4.5 控制风格

集中式控制

少数控制对象掌握大量逻辑。

优点：

- 决策位置集中。
- 流程容易追踪。

缺点：

- 控制器臃肿。
- 其他对象退化为数据容器。

委托式控制

控制器保留主流程，把子任务委托给专业对象。

这是实际项目中较稳妥的风格。

分散式控制

控制逻辑分散在大量小对象中。

问题：

- 控制流难追踪。
- 调试和维护成本高。

4.6 Parnas 信息隐藏

Parnas 的核心思想：

模块隐藏一个重要的设计决策。

模块不只是子程序。模块边界要围绕“未来容易变化的设计决策”设置。

例子：

- 支付平台接口会变化，应隐藏在 `PaymentGateway` 接口背后。
- 数据库表结构会变化，应隐藏在 `Repository` 背后。
- 余额更新事务策略会变化，应隐藏在 `Service` 内部。

4.7 模块化

模块是可以被独立设计、实现和测试的自包含单元。

面向对象中的粒度：

- 方法
- 类
- 包

模块化目标：

- 降低耦合。
- 提高内聚。
- 隐藏变化。
- 支持独立测试。
- 支持并行开发。

4.8 耦合

耦合表示模块之间依赖强度。耦合越低，修改传播越小。

面向对象中两类主要耦合：

- 访问耦合
- 继承耦合

访问耦合

从高到低：

1. 隐式访问耦合：全局变量、单例共享状态。
2. 实现访问耦合：通过反射等方式访问内部实现。
3. 成员变量访问耦合：直接访问 public 字段。
4. 参数变量访问耦合：通过参数传递具体类型。
5. 无访问耦合。

高风险层级：

- 隐式访问耦合
- 实现访问耦合
- 成员变量访问耦合

可接受结构：

- 通过参数和接口协作。
- 不访问对方内部结构。

Law of Demeter

“只和直接朋友交谈”。

一个方法只调用：

- 自身对象的方法。
- 参数对象的方法。
- 本方法创建对象的方法。
- 本对象直接字段的方法。

避免：

```
order.getCustomer().getAddress().getCity()
```

改进方向：

```
order.getCustomerCity()
```

或把责任放到合适对象中。

4.9 接口隔离与面向接口编程

面向接口编程：

- 客户端依赖抽象接口。
- 实现类可以替换。
- 测试时可注入 Mock。

接口隔离原则：

客户端不应被迫依赖它不使用的方法。

坏接口：

```
public interface CampusCardService {  
    void recharge();  
    void reportLost();  
    void generateFinanceReport();  
    void repairDevice();  
}
```

改进：

```
public interface RechargeService {  
    void recharge();  
}  
  
public interface LossReportService {  
    void reportLost();  
}
```

4.10 继承耦合

继承是白盒复用，子类依赖父类内部设计。

继承耦合类型从高到低：

- Modification：子类修改父类语义，风险最高。
- Refinement：子类细化父类行为。
- Extension：子类增加新能力。
- Nil：无继承耦合。

优先选择：

- 组合
- 委托
- 接口

4.11 LSP

LSP 要求：

子类型对象替换父类型对象后，程序正确性保持不变。

契约要求：

- 子类前置条件不能更强。
- 子类后置条件不能更弱。
- 子类必须维护父类不变式。

例子：

- 如果父类 `Account.withdraw(amount)` 允许在余额足够时取款，子类不能额外要求“amount 必须大于 100”。

4.12 组合优于继承

组合通过对象协作复用行为。

优点：

- 运行时可替换。
- 不暴露父类内部实现。
- 更易遵守 LSP。
- 更容易测试。

示例：

- `RechargeService` 组合 `PaymentGateway`，而不是继承某个支付实现类。

4.13 封装

封装层次：

- A：封装数据类型或抽象数据类型。
- B：封装内部结构。
- C：封装其他对象引用。
- D：封装运行时类型信息。
- E：封装潜在变化。

Iterator 模式就是封装集合内部结构，让客户端不用知道底层是 List、Set 还是 Map。

4.14 设计原则 P1-P14

编号	原则	要点
P1	全局变量有害	全局状态导致隐式依赖
P2	显式表达	依赖、状态、约束要清楚
P3	DRY	不重复知识和逻辑
P4	面向接口编程	依赖抽象而非实现
P5	Law of Demeter	只和直接朋友交谈
P6	ISP	接口保持小而专用
P7	LSP	子类可替换父类
P8	组合优于继承	降低白盒复用风险
P9	封装变化	把变化点藏到稳定接口背后
P10	最小权限	只暴露必要访问权限
P11	OCP	对扩展开放，对修改关闭
P12	DIP	高层和低层都依赖抽象
P13	SRP	一个模块只有一个变化理由
P14	SOFA	短、单一、少参数、抽象层一致

SOFA：

- Short
- One thing
- Few arguments
- Abstraction level consistency

4.15 集成测试与 Mock

集成测试检查类之间的协作。

输入：

- 对象组合
- 触发消息

输出：

- 返回值
- 最终对象状态
- 外部协作是否被调用

Mock 对象用于替代外部依赖：

- 数据库
- 支付平台
- 第三方 API
- 消息队列

充值服务测试示例：

- 注入 Mock PaymentGateway 返回支付成功。
 - 注入 Mock CampusCardRepository 返回状态正常的校园卡。
 - 调用 `recharge`。
 - 断言余额更新、记录保存、返回结果正确。
-

5. 设计模式

5.1 设计模式的构成

一个设计模式通常包含：

- 名称
- 问题
- 解决方案
- 应用场景
- 注意事项

模式不是为了“套模板”，而是把稳定抽象和变化点分开。

5.2 Strategy 策略模式

定义：

定义一组算法，把每个算法封装起来，并让它们可以互换，使算法变化独立于使用算法的客户。

结构：

- Context：持有 Strategy。
- Strategy：算法接口。
- ConcreteStrategy：具体算法。

适用场景：

- 同一业务存在多种算法。
- 需要运行时切换算法。
- 代码中存在大量按类型分支选择行为。

例子：充值手续费策略

```
public interface FeeStrategy {
    BigDecimal calculateFee(BigDecimal amount);
}

public class BankCardFeeStrategy implements FeeStrategy {
    public BigDecimal calculateFee(BigDecimal amount) {
        return amount.multiply(new BigDecimal("0.005"));
    }
}

public class AlipayFeeStrategy implements FeeStrategy {
    public BigDecimal calculateFee(BigDecimal amount) {
        return BigDecimal.ZERO;
    }
}
```

Context:

```
public class RechargeServiceImpl implements RechargeService {
    private final FeeStrategy feeStrategy;

    public RechargeServiceImpl(FeeStrategy feeStrategy) {
        this.feeStrategy = feeStrategy;
    }

    public void recharge(BigDecimal amount) {
        BigDecimal fee = feeStrategy.calculateFee(amount);
        // 执行业务流程
    }
}
```

优点:

- 消除复杂条件分支。
- 新增算法时新增策略类。
- 符合 OCP。

代价:

- 类数量增加。
- 客户端需要选择策略。
- Strategy 与 Context 之间需要约定输入输出。

5.3 Abstract Factory 抽象工厂

定义:

提供一个创建一组相关或相互依赖对象的接口，而不指定它们的具体类。

结构:

- AbstractFactory
- ConcreteFactory
- AbstractProduct
- ConcreteProduct
- Client

适用场景:

- 系统需要创建一组相关产品。
- 产品族需要整体替换。
- 客户端不应依赖具体产品类。

例子: 不同平台 UI 组件族

```
public interface UIFactory {
    Button createButton();
    Dialog createDialog();
}

public class WebUIFactory implements UIFactory {
    public Button createButton() {
        return new WebButton();
    }

    public Dialog createDialog() {
        return new WebDialog();
    }
}
```

优点:

- 保证同一产品族一起使用。
- 客户端依赖抽象。
- 切换产品族集中在工厂。

缺点：

- 新增产品种类会修改工厂接口。
- 产品族稳定时效果最好。

5.4 Factory Method 工厂方法

定义：

父类定义创建对象的接口，由子类决定实例化哪一个具体类。

适用场景：

- 父类无法提前确定要创建的具体对象。
- 创建逻辑要延迟到子类。

与抽象工厂区别：

- 工厂方法关注创建一个产品。
- 抽象工厂关注创建一组相关产品。

5.5 Singleton 单例

定义：

保证一个类只有一个实例，并提供全局访问点。

使用时要谨慎：

- 单例容易形成全局状态。
- 会增加隐式耦合。

- 测试替换不方便。

适合：

- 无状态工具入口。
- 全局唯一资源管理器。
- 配置对象。

不适合：

- 承载业务状态。
- 替代依赖注入。

5.6 Iterator 迭代器

定义：

在不暴露聚合对象内部表示的前提下，顺序访问其中元素。

结构：

- Iterator
- ConcreteIterator
- Aggregate
- ConcreteAggregate

优点：

- 隐藏集合内部结构。
- 简化聚合对象接口。
- 支持多种遍历方式。
- 支持多个遍历同时进行。

例子：

客户端使用迭代器遍历充值记录，不关心底层是数组、链表、数据库游标还是分页结果。

6. 代码设计与软件构造

6.1 代码设计关注点

代码设计关注可读性、可维护性和可验证性。

主要内容：

- 布局
- 命名
- 注释
- Javadoc
- 小任务拆分
- 复杂决策表达
- 数据使用
- 显式依赖
- 断言

6.2 布局

原则：

- 缩进表达逻辑层级。
- 空行分隔不同语义块。
- 长表达式拆行。
- 相似语句对齐。
- 复杂控制结构保持清楚。

坏布局会让正确代码难以审查。

6.3 命名

命名要求：

- 表达含义。
- 与领域术语一致。
- 避免误导。
- 避免无意义缩写。
- 避免过长。

Java 约定：

- 包名：小写单词。
- 类名和接口名：PascalCase。
- 变量和方法：camelCase。
- 常量：大写字母加下划线。

语义规则：

- 类、属性、数据对象用名词。
- 接口可用名词或形容词。
- 方法用动词或动词加名词。

示例：

- `RechargeService`
- `CampusCardRepository`
- `findRechargeRecordsByCardNo`
- `MAX_RECHARGE_AMOUNT`

6.4 注释与 Javadoc

Java 注释：

- `//`：单行注释。
- `/* */`：普通块注释。
- `/** */`：文档注释。

Javadoc 常用标签：

- `@author`
- `@version`
- `@see`
- `@since`
- `@deprecated`
- `@param`
- `@return`
- `@throws`

注释原则：

- 解释意图、约束和不直观决策。
- 不重复代码表面含义。
- 公共 API 要说明参数、返回值和异常。

6.5 AI 代码生成 Prompt

四个元素：

- Context：上下文。
- Goal：目标。
- Format：输出格式。

- Steps: 步骤。

示例:

Context:

项目使用 Spring Boot, 分层为 controller/service/repository。

Goal:

实现校园卡充值服务接口和实现类。

Format:

输出 Java 接口、实现类和单元测试。

Steps:

先定义 DTO, 再定义 Service 接口, 再写实现逻辑, 最后写测试。

6.6 软件构造

SWEBOK 中软件构造覆盖:

- 编码
- 验证
- 单元测试
- 集成测试
- 调试

构造不是“写代码”一个动作, 而是把设计转化为可运行、可验证、可维护的软件制品。

6.7 构造技术

主要技术:

- 可理解的源代码
- 类、枚举、变量、常量
- 控制结构
- 错误处理

- 代码级安全
- 单元测试
- 集成测试
- 调试

6.8 代码审查

课程给出的审查经验：

- 每次审查少于 200 到 400 行代码。
- 审查速度控制在每小时 300 到 500 行。
- 单次审查时间控制在 60 到 90 分钟。
- 开发者先对代码做注解。
- 使用检查表。
- 确认缺陷已修复。
- 保持积极的审查文化。
- 借助工具支持轻量化审查。

AI 辅助代码审查适合：

- 识别坏味道。
- 检查安全问题。
- 检查风格问题。
- 发现遗漏测试。

最终结论要由人结合需求、设计和上下文确认。

6.9 集成与构建

集成策略：

- 大爆炸集成：一次性集成所有模块。
- 增量集成：逐步集成并验证模块。

增量集成更利于定位问题和控制风险。

构建：

将源代码、资源、配置和依赖转化为可执行或可部署制品。

构建依赖配置管理：

- 版本一致。
- 依赖可追踪。
- 构建脚本受控。
- 发布制品可复现。

6.10 构造管理

构造管理内容：

- 分配构造任务。
- 跟踪代码进度。
- 度量复杂度。
- 度量代码行数。
- 度量注释比例。
- 管理分支、合并和发布。
- 执行代码审查和测试门禁。

6.11 坏味道与重构

重构定义：

在不改变软件外部行为的前提下，改善内部结构。

常见坏味道：

- 重复代码。
- 长方法。
- 大类。
- 长参数列表。
- Magic Number 或 Magic String。
- 空 catch。
- 过多注释。
- 某个类过度访问其他类属性。
- Switch 或 if-else 按类型分发行为。

对应改进：

坏味道	问题	重构
重复代码	隐式耦合	提取方法或提取类
长方法	难理解、难测试	提取方法
大类	职责过多	提取类、按职责拆分
长参数列表	调用复杂	引入参数对象
Magic Number	含义不清	提取常量
空 catch	吞掉错误	记录日志或抛出业务异常
过多注释	结构表达力不足	改名、拆分、重组逻辑
过度访问其他类属性	职责位置错误	移动方法或委托
类型分支	违反 OCP	策略模式或多态

6.12 TDD

TDD 循环：

1. Red：先写失败测试。
2. Green：写最少代码让测试通过。
3. Refactor：在测试保护下重构。

适合：

- 规则明确的业务逻辑。
- 算法。
- 服务层。
- 状态迁移。

6.13 AI 编程原则

使用 AI 写代码时：

- 明确意图。
- 逐步细化。
- 保持对代码的理解。
- 先定义测试或验收标准。
- 经过验证后再提交。

不能把 AI 输出直接当作正确实现。需要用需求、设计、测试和代码审查约束它。

7. 综合题答题模板

7.1 分层接口代码模板

题型：

给定某个情境，写出 Service 接口声明、Repository 方法声明、Controller 调用 Service 的代码片段，并体现依赖注入。

作答要求：

- 写包名。
- 写接口。
- 写构造器注入。
- Controller 调用 Service。
- ServiceImpl 调用 Repository。
- Controller 不直接调用 Repository。

示例：校园卡充值

```
package edu.school.card.service;

import java.math.BigDecimal;
import java.util.List;

public interface RechargeService {
    RechargeResult recharge(RechargeCommand command);

    List<RechargeRecordView> findRechargeRecords(String cardNo);
}

package edu.school.card.repository;

import java.util.List;
import java.util.Optional;

public interface CampusCardRepository {
    Optional<CampusCard> findByCardNo(String cardNo);
}
```

```
        void save(CampusCard campusCard);
    }

package edu.school.card.repository;

import java.util.List;

public interface RechargeRecordRepository {
    void save(RechargeRecord rechargeRecord);

    List<RechargeRecord> findByCardNo(String cardNo);
}

package edu.school.card.controller;

import edu.school.card.service.RechargeCommand;
import edu.school.card.service.RechargeResult;
import edu.school.card.service.RechargeService;
import edu.school.card.service.RechargeRecordView;
import java.util.List;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api/recharges")
public class RechargeController {
    private final RechargeService rechargeService;

    public RechargeController(RechargeService rechargeService) {
        this.rechargeService = rechargeService;
    }

    @PostMapping
    public RechargeResult recharge(@RequestBody RechargeCommand command) {
        return rechargeService.recharge(command);
    }

    @GetMapping("/cards/{cardNo}")
    public List<RechargeRecordView> findRechargeRecords(
```

```
        @PathVariable String cardNo) {
            return rechargeService.findRechargeRecords(cardNo);
        }
    }

package edu.school.card.service.impl;

import edu.school.card.repository.CampusCardRepository;
import edu.school.card.repository.RechargeRecordRepository;
import edu.school.card.service.RechargeCommand;
import edu.school.card.service.RechargeRecordView;
import edu.school.card.service.RechargeResult;
import edu.school.card.service.RechargeService;
import java.util.List;

public class RechargeServiceImpl implements RechargeService {
    private final CampusCardRepository campusCardRepository;
    private final RechargeRecordRepository rechargeRecordRepository;

    public RechargeServiceImpl(
        CampusCardRepository campusCardRepository,
        RechargeRecordRepository rechargeRecordRepository) {
        this.campusCardRepository = campusCardRepository;
        this.rechargeRecordRepository = rechargeRecordRepository;
    }

    @Override
    public RechargeResult recharge(RechargeCommand command) {
        CampusCard card = campusCardRepository.findByCardNo(command.cardNo())
            .orElseThrow(CardNotFoundException::new);
        card.recharge(command.amount());
        RechargeRecord record = RechargeRecord.success(
            command.cardNo(),
            command.amount());
        campusCardRepository.save(card);
        rechargeRecordRepository.save(record);
        return RechargeResult.success(record.id());
    }

    @Override
    public List<RechargeRecordView> findRechargeRecords(String cardNo) {
        return rechargeRecordRepository.findByCardNo(cardNo).stream()
            .map(RechargeRecordView::from)
            .toList();
    }
}
```

```
}  
}
```

答题说明:

- `controller` 是表示层入口。
- `service` 是表示层与逻辑层之间的接口。
- `repository` 是逻辑层与数据层之间的接口。
- 构造器注入体现依赖注入。
- 代码里出现的 DTO、Entity 和异常类可以只声明名称，重点是层间依赖方向。

7.2 校园卡自助系统物理包图文字版

题型:

画出校园卡自助系统物理包图。分为客户端和服务端，体现分层概念，包含所有模块。三包和展示层在客户端，逻辑层和数据层在服务端，使用 HTTP 通信和 REST API。

考试作图要素:

- 客户端节点。
- 服务器端节点。
- 客户端包含 HTML、CSS、JavaScript 三包。
- 展示层在客户端。
- 逻辑层和数据层在服务端。
- 客户端与服务器通过 HTTP 通信。
- 通信接口是 REST API。
- 服务器端内部体现 Controller、Service、Repository、数据库。

文字图:

```
<<device>> Client
package html
package css
package javascript
package presentation
    module LoginPage
    module RechargePage
    module RechargeRecordPage
    module LossReportPage
    module BalanceQueryPage
    module ProfilePage

presentation --HTTP/REST API--> server.api

<<server>> Server
package api
    module AuthController
    module RechargeController
    module BalanceController
    module LossReportController
    module CardController

package service
    interface AuthService
    interface RechargeService
    interface BalanceService
    interface LossReportService
    interface CardService

package service.impl
    class AuthServiceImpl
    class RechargeServiceImpl
    class BalanceServiceImpl
    class LossReportServiceImpl
    class CardServiceImpl

package repository
    interface UserRepository
    interface CampusCardRepository
    interface RechargeRecordRepository
    interface LossReportRepository
    interface TransactionRepository

package repository.impl
```

```
class UserRepositoryImpl
class CampusCardRepositoryImpl
class RechargeRecordRepositoryImpl
class LossReportRepositoryImpl
class TransactionRepositoryImpl

package domain
class User
class CampusCard
class RechargeRecord
class LossReport
class Transaction

package database
table user
table campus_card
table recharge_record
table loss_report
table transaction
```

依赖方向:

```
presentation -> api -> service -> repository -> database
service.impl -> domain
repository.impl -> domain
```

图上必须体现:

```
Client.presentation -- HTTP + REST API --> Server.api
```

7.3 充值服务接口题模板

题型:

针对充值服务, 包括充值、查询充值记录, 写出展示层和逻辑层之间接口, 以及逻辑层和数据层之间接口。注意写出接口所在包名。

展示层和逻辑层之间接口:

```
package edu.school.card.service;
```

```
import java.util.List;

public interface RechargeService {
    RechargeResult recharge(RechargeCommand command);

    List<RechargeRecordView> findRechargeRecords(String cardNo);
}
```

逻辑层和数据层之间接口:

```
package edu.school.card.repository;

import java.util.Optional;

public interface CampusCardRepository {
    Optional<CampusCard> findByCardNo(String cardNo);

    void save(CampusCard campusCard);
}

package edu.school.card.repository;

import java.util.List;

public interface RechargeRecordRepository {
    void save(RechargeRecord rechargeRecord);

    List<RechargeRecord> findByCardNo(String cardNo);
}
```

DTO 示例:

```
package edu.school.card.service;

import java.math.BigDecimal;

public record RechargeCommand(String cardNo, BigDecimal amount, String payChannel) {
}

package edu.school.card.service;

public record RechargeResult(String recordId, String status, String message) {
}
```

```
package edu.school.card.service;

import java.math.BigDecimal;
import java.time.LocalDateTime;

public record RechargeRecordView(
    String recordId,
    String cardNo,
    BigDecimal amount,
    String status,
    LocalDateTime createdAt) {
}
```

答题解释：

- RechargeService 是展示层调用逻辑层的接口。
- CampusCardRepository 和 RechargeRecordRepository 是逻辑层调用数据层的接口。
- RechargeService 不暴露数据库表结构。
- Repository 不处理充值业务规则，只负责数据读取和保存。

7.4 需求题答题模板

写业务需求

BR-01 建设校园卡自助系统，提高校园卡充值、查询、挂失等业务的自助办理比例，减少人工窗口处理压力。

写用户需求

UR-01 作为持卡人，我想在线充值校园卡，以便在余额不足时继续使用校园卡。

写系统级功能需求

FR-01 当持卡人提交充值请求时，系统应校验持卡人身份、校园卡状态和充值金额。

FR-02 当支付平台返回支付成功结果时，系统应更新校园卡余额并生成成功充值记录。

FR-03 当持卡人查询充值记录时，系统应返回该校园卡的充值记录列表。

写非功能需求

NFR-01 系统应通过 HTTPS 提供充值接口。

NFR-02 充值成功响应时间应不超过 3 秒，不包含第三方支付平台处理时间。

NFR-03 系统应记录每次充值请求、支付结果和余额更新结果。

7.5 概念类图题模板

校园卡系统概念类：

- User
- CampusCard
- RechargeRecord
- PaymentOrder
- LossReport
- Transaction

主要关系：

- 一个 User 拥有零张或多张 CampusCard。
- 一张 CampusCard 对应零条或多条 RechargeRecord。
- 一条 RechargeRecord 对应一个 PaymentOrder。
- 一张 CampusCard 对应零条或多条 LossReport。
- 一张 CampusCard 对应零条或多条 Transaction。

概念类示例：

User

userId
name
studentNo

CampusCard

cardNo
balance

```
status
```

```
RechargeRecord
```

```
recordId  
amount  
status  
createdAt
```

```
PaymentOrder
```

```
orderId  
channel  
status  
paidAt
```

7.6 顺序图题模板

充值基本流程：

```
Actor -> RechargePage: 输入金额并提交  
RechargePage -> RechargeController: POST /api/recharges  
RechargeController -> RechargeService: recharge(command)  
RechargeService -> CampusCardRepository: findByCardNo(cardNo)  
CampusCardRepository --> RechargeService: CampusCard  
RechargeService -> PaymentGateway: pay(order)  
PaymentGateway --> RechargeService: PaymentResult  
RechargeService -> CampusCard: recharge(amount)  
RechargeService -> CampusCardRepository: save(card)  
RechargeService -> RechargeRecordRepository: save(record)  
RechargeService --> RechargeController: RechargeResult  
RechargeController --> RechargePage: JSON result
```

异常分支：

- 校园卡不存在：抛出 `CardNotFoundException`。
- 校园卡已挂失：返回充值拒绝。
- 支付失败：保存失败记录并返回失败原因。
- 保存余额失败：执行事务回滚并记录错误。

7.7 状态图题模板

校园卡状态:

```
Normal -> Lost: reportLost
Lost -> Normal: cancelLossReport
Normal -> Frozen: freeze
Frozen -> Normal: unfreeze
Normal -> Cancelled: cancel
Lost -> Cancelled: cancel
Frozen -> Cancelled: cancel
```

充值记录状态:

```
Created -> Paying: startPayment
Paying -> Success: paymentSucceeded
Paying -> Failed: paymentFailed
Created -> Cancelled: cancel
Failed -> Created: retry
```

测试点:

- 正常迁移要测。
- 非法迁移要测。
- 每个状态的允许操作要测。

7.8 设计原则题模板

题目给出代码后, 按以下结构答:

1. 指出问题。
2. 对应原则。
3. 说明风险。
4. 给出改进。

示例:

问题: Controller 直接访问 Repository, 并在 Controller 中完成充值业务规则。

原则: 违反分层职责、低耦合和 SRP。

风险: 界面入口与数据库访问耦合, 业务规则无法复用, 单元测试困难。

改进: Controller 只依赖 RechargeService;

RechargeServiceImpl 注入 Repository 并封装充值流程。

7.9 设计模式题模板

题目出现大量按类型选择算法:

```
if (payType.equals("BANK")) { ... }  
else if (payType.equals("ALIPAY")) { ... }  
else if (payType.equals("WECHAT")) { ... }
```

答:

- 使用 Strategy。
- 抽象 PaymentStrategy。
- 每种支付方式一个具体策略。
- RechargeService 依赖 PaymentStrategy 或策略工厂。
- 新增支付方式时新增策略类, 减少修改原业务流程。

题目出现一组相关产品整体替换:

- 使用 Abstract Factory。
- 抽象产品和抽象工厂。
- 具体工厂创建同一产品族。

题目要求隐藏集合结构:

- 使用 Iterator。
- 客户端通过迭代器访问元素。

7.10 代码质量题模板

答题结构:

问题:

违反原则:

影响:

重构:

测试:

示例:

问题: 充值方法过长, 同时处理参数校验、支付调用、余额更新、记录保存和异常处理。

违反原则: SRP、SOFA。

影响: 代码难理解、难测试、修改支付逻辑会影响余额更新逻辑。

重构: 提取校验方法、支付网关、余额更新方法和记录保存方法; 支付方式变化使用 Strategy。

测试: 为正常充值、支付失败、校园卡挂失、金额非法分别编写单元测试。

7.11 HCI 题模板

题目要求评价界面:

问题: 充值按钮太小且远离金额输入区域。

原则: 违反 Fitts 定律和效率原则。

影响: 用户完成高频充值操作的时间增加, 误触风险上升。

改进: 增大按钮点击区域, 把提交按钮放在金额输入区域附近, 并对危险操作使用确认反馈。

题目要求设计反馈:

充值提交后, 系统立即显示处理中状态;

支付成功后显示余额变化和充值记录编号;

支付失败时显示失败原因和重试入口;

长时间等待时显示进度或可取消操作。

7.12 配置管理题模板

题目出现上线后版本不一致、漏部署、旧代码被触发:

问题属于配置管理和发布管理失败。

原因: 配置项识别不完整, 版本状态未核对, 发布前未做配置审计和上线验证。

改进:

1. 将源码、脚本、配置、依赖和部署清单纳入配置项。
 2. 建立发布基线。
 3. 变更走 CR、影响分析、CCB 审批、执行、审计、状态报告。
 4. 发布前核对所有服务器版本。
 5. 发布后执行冒烟测试和回滚预案。
-

8. 快速背诵清单

软件工程

- 软件 = 程序 + 数据 + 文档 + 知识。
- 软件工程 = 系统化、规范化、可度量的方法。
- 软件危机来自规模、复杂度、变化和协作失控。
- AI 加速工程活动，但不能替代验证、评审和配置管理。

项目管理

- 项目：临时性、独特性。
- 三约束：范围、时间、成本，影响质量。
- 五过程组：启动、计划、执行、跟踪控制、收尾。
- 风险管理：识别、分析、排序、应对、监控。
- SCM：配置项、基线、版本、变更、审计、状态报告、发布。

需求工程

- 活动：获取、分析、规格说明、验证、管理。
- 层次：业务需求、用户需求、系统级需求。
- 类型：功能、非功能、接口、约束、数据。
- 用例：用户目标、参与者、系统边界、include、extend。
- SRS：系统视角、刺激-响应、可验证、可追踪。

体系结构

- 架构 = 组件 + 连接件 + 配置。
- 4+1: 逻辑、开发、进程、物理、场景。
- 分层: 表示层、业务逻辑层、数据访问层。
- 依赖倒置: 上层依赖接口, 不依赖实现。

HCI

- 可用性: 易学、效率、易记、错误恢复、满意度。
- Fitts: 目标越近越大, 操作越快。
- 识别优于回忆。
- 黄金规则: 用户控制、减少记忆负担、一致性。

详细设计

- 核心: 责任与协作。
- GRASP: 低耦合、高内聚、信息专家、创建者、控制器。
- 信息隐藏: 模块隐藏重要设计决策。
- SOFA: 短、单一、少参数、抽象层一致。

设计模式

- Strategy: 封装可互换算法。
- Abstract Factory: 创建一组相关产品。
- Factory Method: 子类决定创建哪个产品。
- Singleton: 唯一实例和全局访问点。
- Iterator: 隐藏集合内部结构并顺序访问元素。

构造

- 构造 = 编码 + 验证 + 单元测试 + 集成测试 + 调试。
- 代码审查要小批量、限时、用检查表。
- 重构不改变外部行为，只改善内部结构。
- TDD: Red、Green、Refactor。